

CS69021: Computing Lab-1
Test-6 || August 17, 2026
Time: 2.5 hours

===== **Instructions** =====

1. Q1 carries **25 marks**, Q2 carries **25 marks**, Q3 carries **30 marks** and Q4 carries **40 marks**.
2. ***Make sure to follow input/output patterns. The submitted code will be evaluated automatically against a series of hidden test cases.***
3. You can use C or C++ as the programming language. **You are allowed to use any STL libraries in C++ for this test.**
4. Submission Guidelines
 - a. First, rename the folder appropriately. If your roll number is 26CS60R15 and seat number is 10 then the folder name should be **26CS60R15_10_T6**.
 - b. Second, choose in which language you want to code. If you choose the **CPP** folder, delete the **C** folder and vice-versa. **[Don't delete the header folder]**
 - c. Third, rename the file names appropriately. If your roll number is 26CS60R15 and seat number is 10 then the question name should be **26CS60R15_10_Q1.cpp**, **26CS60R15_10_Q2.cpp**, **26CS60R15_10_Q3.cpp**, **26CS60R15_10_Q4.cpp**,
 - d. At the time of submission you have to zip the **26CS60R15_10_T6** folder and submit the **26CS60R15_T6.zip** file.
5. **Plagiarism in any form is strictly prohibited. If detected, it may result in a significant grade penalty.**

=====

Q1) THE JARL'S BEACONS

You are the High Chieftain of a vast Norse clan. Your people have settled along a treacherous, branching river system in the frozen north. To maintain order, your settlements are structured in a **strict Binary Hierarchy (a binary tree)**, where each outpost oversees **at most two subordinate outposts** further down the river.

However, news travels in all directions: **messengers can sail downriver to subordinate outposts, or hike upriver to their direct commanding outpost**. Travel between any two directly connected outposts takes **exactly 1 day**.

Disaster has struck! A rival clan has laid siege to the outpost designated as the **target**. You must summon reinforcements, but you want them all to arrive on the exact same day to coordinate a massive flank attack. You need to send ravens to all outposts that are exactly **k days** of travel away from the **target**.

The Task ~

Given the **root** of the binary tree of outposts, the **target** outpost node, and an integer **k** (representing the days of travel), **return an array of the values (outpost IDs)** of all outposts that have a travel distance of exactly **k** from the target node.

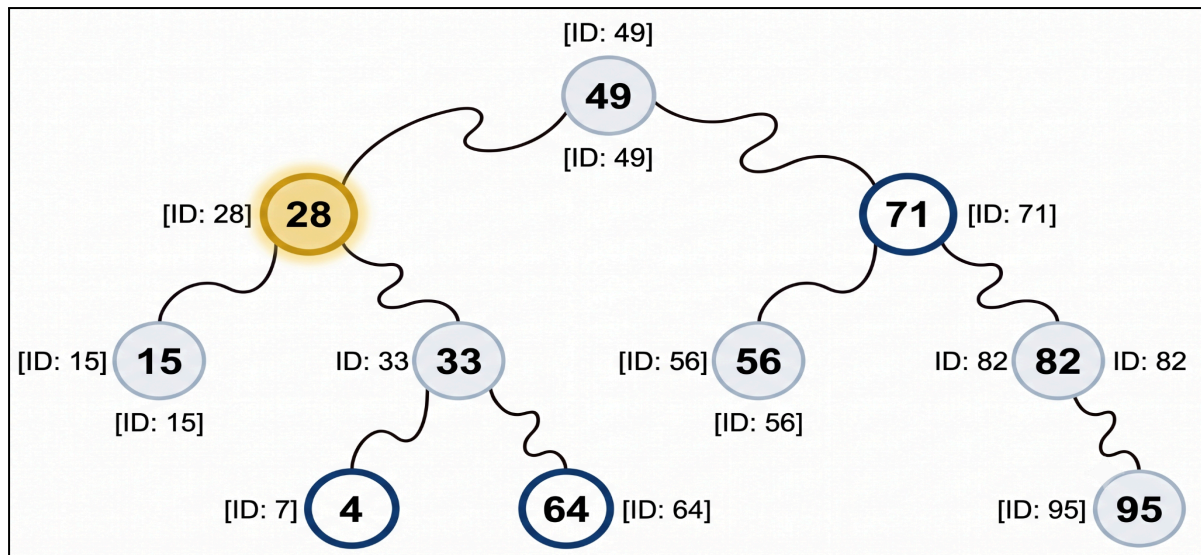
You must return the answer in sorted order.

Constraints ~

1. The number of outposts (nodes) in the tree is in the range [1, 500].
2. $0 \leq \text{Node.val} \leq 500$
3. All the values in Node.val are unique.
4. target is guaranteed to be a valid outpost in the network.
5. $0 \leq k \leq 1000$

Examples ~

n = 15 ,root = [49, 28, 71, 15, 33, 56, 82, -1, -1, 4, 64, -1, -1, -1, 95] target = 28, k = 2	[4, 64, 71]	The outpost with ID 28 is under siege. You need to find all outposts in exactly 3 days of travel. - (5 -> 2 -> 7). - (5 -> 2 -> 4). - (5 -> 3 -> 1).
n = 1, root = [1], target = 1, k = 3	[]	You only have one outpost. There is no other way to send ravens to. You must face the siege alone.



Q2) THE K-TH NEAREST ENEMY LONGSHIP

You are the Master Navigator stationed at the high watchtower of Kattegat, located at the very center of the realm at **coordinates (0, 0)**. The frozen sea stretches endlessly in all directions.

Through the thick, freezing fog, your scouts continuously shout down reports of rival clan longships dropping anchor at various coordinates. Because the sea is choked with a rigid grid of massive ice floes, your warriors cannot sail in a straight line. The sailing distance to any enemy ship located at **(x, y)** must be calculated using the **square of Euclidean Distance = (x² + y²)**.

The Jarl is preparing a defensive perimeter. He will not order the archers to lose their arrows at the closest ship, nor the furthest. For his tactical formation to work, he needs to constantly know **exact distance** to the **k-th nearest enemy longship** currently in the waters.

The Task ~

You are given an integer **k** and a 2D array queries, where **queries[i] = {x, y}** represents the coordinates of a newly spotted rival longship.

For every longship reported, you must shout back the square of the distance of the **k-th nearest longship** spotted so far. If there are currently fewer than **k** longships in the water, you must shout **-1** to tell the Jarl to hold his command.

Return an array of integers representing the distance you report to the Jarl after each scout's query.

Example 1 ~

Input: $k = 2$, $q = 4$, queries = $[[1, 2], [3, 4], [2, 3], [-3, 0]]$

Output: $[-1, 25, 13, 9]$

Explanation: The Jarl wants to know the distance to the **2nd** nearest ship ($k = 2$).

- 1) A ship is spotted at $[1, 2]$. Its sailing distance is $(1^2 + 2^2) = 5$.
 - There is only 1 ship, so you shout **-1**.
- 2) A ship is spotted at $[3, 4]$. Its distance is $(3^2 + 4^2) = 25$.
 - The 2nd nearest ship is at distance **25**.
- 3) A ship is spotted at $[2, 3]$. Its distance is $(2^2 + 3^2) = 13$.
 - The 2nd nearest ship is at distance **13**.
- 4) A ship is spotted at $[-3, 0]$. Its distance is $((-3)^2 + 0^2) = 9$.
 - The 2nd nearest ship is at distance **9**.

Example 2 ~

Input: $k = 1$, $q = 3$, queries = $[[5, 5], [4, 4], [3, 3]]$

Output: $[50, 32, 18]$

Explanation: The Jarl wants to know the distance to the **1st** nearest ship. and answer is 32

- 1) Ship spotted at $[5, 5]$. The distance is **50**. The closest ship is distance **50**.
- 2) Ship spotted at $[4, 4]$. The distance is **32**. The closest ship is now distance **32**.
- 3) Ship spotted at $[3, 3]$. The distance is **18**. The closest ship is now distance **18**.

Constraints ~

1. $1 \leq \text{queries.length} \leq 2 * 10^5$
2. All queries[i] are **unique** (no two longships anchor at the exact same coordinates).
3. $-10^9 \leq \text{queries}[i][0], \text{queries}[i][1] \leq 10^9$
4. $1 \leq k \leq 10^5$

Q3> THE VIKINGS SCOUTS

After a fierce **Battle of Paris**, **Ragnar Lothbrok** wants to make sure his entire settlement is protected from enemy attacks so he asks **Floki** to send scouts to watch over the entire settlement.

The settlement is organized like a **binary tree**, where each Viking camp is represented by a node. Every camp can have at most two neighboring camps: a left camp and a right camp.

Floki can place a **scout** at any camp. A scout can keep watch over:

- The camp where they are stationed
- Its parent camp
- Its immediate left and right neighboring camps

Floki wants to use the **minimum number of scouts** while ensuring that **every camp in the settlement is being watched by at least one scout**.

The Task ~

Given the root of the binary-tree representation of the Viking settlement, **return the minimum number of scouts Floki needs to place to protect the entire settlement**.

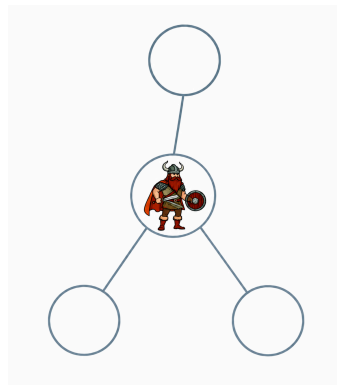
Constraints ~

- The number of nodes in the tree is in the range [1,1000].
- `node.val = 0`

Example 1~

Input: `n = 5, root = [0, 0, -1, 0, 0]`

Output: 1

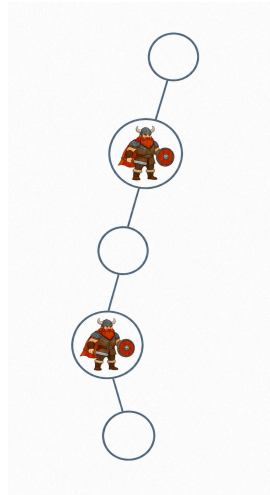


Explanation: Floki can place one scout at a suitable Viking camp to monitor all the camps in the settlement.

Example 2~

Input: `n = 9, root = [0, 0, -1, 0, -1, 0, -1, -1, 0]`

Output: 2



Explanation: Floki needs at least two scouts to monitor all the camps in the settlement. The above arrangement shows one possible way to place the two scouts.

Q4) THE DIVISION OF KATTEGAT

After a fierce battle in **Kattegat**, **Lagertha** and **Bjorn Ironside** must divide the conquered territory into separate regions.

The territory consists of **n settlements**, represented as the **nodes of a tree**, connected by exactly **$n - 1$ paths (edges)**. Since the paths form a tree, there is exactly one path between any two settlements.

Bjorn wants to divide the territory by destroying **exactly k paths**. However, Lagertha insists that the division must be fair: **every resulting region must contain at least x settlements**.

The Task ~

Find the **maximum possible value of x** such that Bjorn can destroy exactly **k paths** and ensure that every remaining connected region contains at least **x settlements**.

Input Format ~

The first line contains two integers n and k , the number of settlements and the number of paths that must be destroyed.

Each of the next $n - 1$ lines contains two integers v and u , indicating that settlements v and u are connected by a path.

It is guaranteed that the sum of n over all test cases does not exceed 10^5 .

Constraints ~

- $1 \leq k < n \leq 10^5$

- Each Node represented by Unique value from 1 to n (both inclusive)

Output Format ~

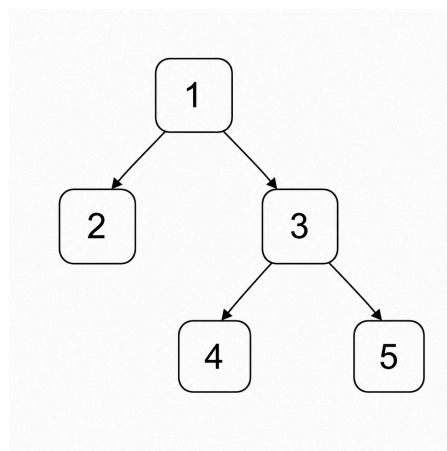
For each territory, output a single integer **x**, the maximum possible minimum number of settlements in any remaining region after exactly **k** paths are destroyed.

Example 1 ~

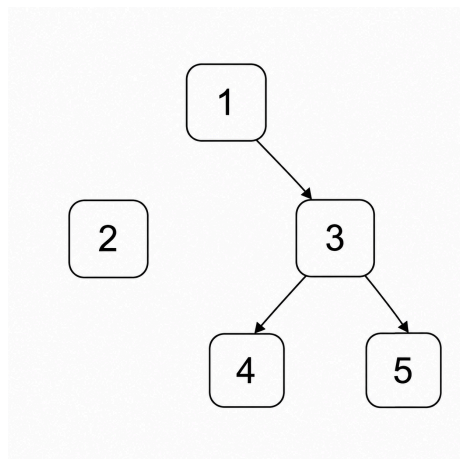
Input: $n = 5, k = 1, \text{adj} = [[1, 2], [1, 3], [3, 4], [3, 5]]$

Output: 2

Explanation: There are 5 settlements in Kattegat and Bjorn must destroy exactly 1 path. The territory looks like:



Bjorn can destroy the path between 1 and 2:

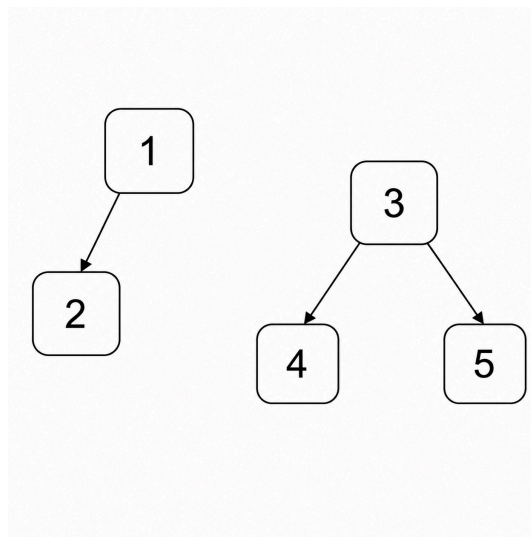


This creates two regions:

- Region 1 $\rightarrow \{2\} \rightarrow 1$ settlement
- Region 2 $\rightarrow \{1, 3, 4, 5\} \rightarrow 4$ settlements

So the minimum region size is 1.

Also, Bjorn can destroy the path between 1 and 3:



Now the regions contain:

- $\{1, 2\} \rightarrow 2$ settlements
- $\{3, 4, 5\} \rightarrow 3$ settlements

So, the minimum region has at least 2 settlements therefore answer = 2.

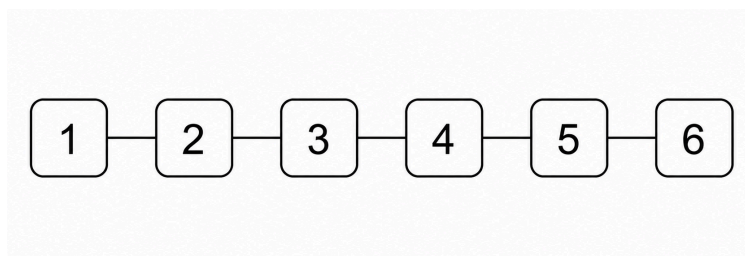
Example 2 ~

Input : $n = 6, k = 1, \text{adj} = [[1, 2], [2, 3], [3, 4], [4, 5], [5, 6]]$

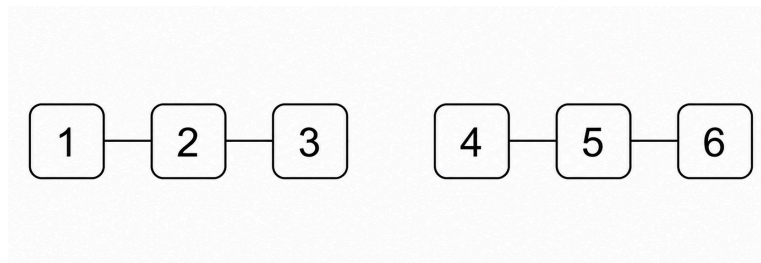
Output : 1

Explanation: There are 6 settlements in Kattegat and Bjorn must destroy exactly 1 path.

The territory looks like:



Bjorn can destroy the path between 3 and 4:



This creates two regions:

- Region 1 $\rightarrow \{1, 2, 3\} \rightarrow 3$ settlements
- Region 2 $\rightarrow \{4, 5, 6\} \rightarrow 3$ settlements

So, every region contains at least 3 settlements. Therefore Answer = 3.